

Theory:

1) The basic arithmetic operators are:

- i) $+$ $\rightarrow$ Addition
- ii) $-$ $\rightarrow$ Subtraction
- iii) $*$ $\rightarrow$ Multiplication
- iv) $/$ $\rightarrow$ Division
- v) $\%$ $\rightarrow$ Remainder (Modulus)
- vi) $**$ $\rightarrow$ Exponentiation (Power)

2) $*$ = assigns a value to a variable

$* +=$ adds a value to the existing variable and assigns the result back.

$* ++$ increase a variable's value by 1.

Eg) $\text{let } x = 5;$
 $x = 10;$
 $x += 5;$
 $x ++;$

3) $\&\&$ returns the first falsy value, or the last value if all values are truthy.

$\|\|$ returns the first truthy value, or the value if all values are falsy. They return an operand's actual value, not necessarily true or false.

4) The ternary operator is a short form of if/else. It checks a condition and returns one of two values. It is preferable when the condition is simple and only two possible results are required.

Syntax:

Condition ? valueIf true : valueIf false

5) The $??$ operator provides a fallback value only when the left side is null or undefined.

The $||$ operator provides a fallback when the left side is

any falsy value, such as 0, "", false, null, or undefined.

Eg) $0 \&\&\& 10 \rightarrow 0$
 $0 || 10 \rightarrow 10$

6) Optional chaining ?. is used to safely access properties or methods that may not exist. If the property or object is null or undefined, it returns undefined instead of causing an error.

7) The spread operator is used to expand or copy elements/properties. With arrays, it can be used to combine or copy arrays. With objects, it can be used to combine or copy object properties.

8) The rest parameter collects multiple function arguments into a single array. Difference:

Spread $\rightarrow$ expand / ~~up~~ unpacks values

Rest $\rightarrow$ Collects multiple values. The rest parameter must be the last parameter in a function.

9) Operator precedence determines the order in which operators are evaluated. Multiplication has higher precedence than addition. Therefore: $2 + 3 * 4$

$$= 2 + 12$$
$$= 14$$

It does not equal 20 because addition is not performed first.

10) A unary operator operates on only one operand.

Eg):

$* + x$

$* - x$

Other examples include $!x$, $++x$, and $typeof x$.

Output prediction:

11) 9

12) 14

13) 1

14) 8

15) hello

16) default

17) fallback

18) 0

19) fallback

20) undefined

Error Finding

21) There is no error. The result is: Infinity

22) There is no error, but the result is: NaN

0/0 does not produce a meaningful numeric result, so JavaScript returns NaN.

23) This produces a TypeError because you cannot access a property from null.

Eg): `console.log(obj?.name);`

This returns:

undefined

24) There is no error

? ? = assigns the value on the right only when the variable is null or undefined. Since x is initially undefined, the value becomes 10. O/P: 10

25) 3. The - operator converts the numeric string into numbers before performing subtraction.

26) true.

Reason: First: $1 < 2 \Rightarrow \text{true}$

Then JavaScript converts true to 1:

$1 < 3 \rightarrow \text{true}$. Therefore the final result is true.

27) False.

Reason

First: $3 > 2 \rightarrow \text{true}$

Then true becomes 1:

$1 > 1 \rightarrow \text{false}$. Therefore the final result is false.

28) There is no error.

There is no fourth element in the array, so d receives: undefined

29) This produces a Syntax Error. The rest parameters must always be the last parameter in a function, correct eg):
function sum(first, ...nums) { }

30) false.

NaN is not equal to itself using $==$ or $===$.

To check for NaN, use:

Number.isNaN(value)

Coding:

31) `console.log(10 + 5 * 2 - 4)` O/P: 16

32) `let age = 20;`

`let status = age >= 18 ? "Adult" : "Minor";`

`console.log(status);` O/P: Adult

33) `let value = 0;`

`let result = value ?? "Default";`

`console.log(result)` O/P = 0

```

34) const user = {
  profile: {},
  settings: {
    theme: "dark"
  }
};

```

```

console.log(user?.profile?.settings?.theme);

```

O/P: dark

```

35) const arr1 = [1, 2, 3];
    const arr2 = [4, 5, 6];
    const merged = [...arr1, ...arr2];
    console.log(merged);

```

O/P: [1, 2, 3, 4, 5, 6]

```

36) const obj1 = {name: "John", age: 20};
    const obj2 = {age: 25, city: "Chennai"};
    const merged = {...obj1, ...obj2};
    console.log(merged);

```

Result:

```

{
  name: "John",
  age: 25,
  city: "Chennai"
}

```

```

37) function sum(...numbers) {
  return numbers.reduce((total, number) => total + number, 0);
}
console.log(sum(10, 20, 30));

```

O/P: 60

38) function refresh() {

console.log("Refreshed");

}

let isOnline = true;

isOnline & refresh(); O/P: Refreshed.

39) Without parentheses:

console.log(10+5*2); O/P: 20

With parentheses

console.log((10+5)*2); O/P: 30

40) let price = "100";

let total = +price 50;

console.log(total); O/P: 150

Real World scenario;

41) ??

?? is appropriate because it provides the real name only when the nickname is null or undefined. It does not treat an empty string as missing, so an intentionally empty nickname is preserved.

42) Use optional chaining ?.

It safely access the nested properties without throwing an error if profile or settings does not exist.

43) use ?? ~~into~~ instead of ||.

0 is a valid value but it is falsy. || would replace it with the default value, while ?? keep 0.

44) Use the spread operator...

It allows the base configuration and user specific properties to be combined easily.

45) use a rest parameter (...). It allows a function to accept any number of arguments and collect them into an array.

Eg): function createPost(...tags) {

// tags is an array
}

46) Use the logical AND (&) operator

isOnline & refresh();

The refresh() function is called only when isOnline is truthy.

47) parentheses explicitly control the order of operations and ensure that the discount is applied before the tax. Without parentheses, JavaScript's operator precedence may evaluate the expression in a different order and produce an incorrect result.

48) Use the spread operator (...) placing the default settings first and the user's settings second:

{...defaultSettings, ...userSettings}

properties in userSettings override matching properties in defaultSettings because they appear later.

49) The == operator performs type coercion, meaning JavaScript may convert values to the same type before comparing them. This can cause different values or types

to be considered equal unexpectedly, leading to incorrect validation results. A clear strict comparison or an appropriate ternary/nullish check makes the intended condition explicit.

50) The optional chaining operator($?.$) safely access the API field even if an intermediate property is missing. The nullish Coalescing Operator($??$) provides a fallback when the resulting value is null or undefined. An empty string ($""$) is not considered nullish, so $??$ preserves it.

For eg):

```
const name = response?.data?.name ?? "No name provided";
```

The behavior is

Missing response $\rightarrow$ "No name provided"

Missing data $\rightarrow$ "No name provided"

Missing name $\rightarrow$ "No name provided"

~~name~~ Missing = null $\rightarrow$ "No ~~is~~ name provided"

~~name~~ name = "" $\rightarrow$ "" is preserved.

This combination makes the code safer when handling incomplete or inconsistent API data.